

BPF-EXT: Safely Extending the eBPF Compilation Pipeline with Native Operations

Anonymous Author(s)

Abstract

eBPF safely extends OS kernels in domains such as networking, observability, and security. The safety comes from an in-kernel compilation pipeline: a verifier checks every program for safety, and a kernel just-in-time compiler (JIT) translates the verified bytecode to native code. The kernel keeps the JIT simple to stay trustworthy; it translates one instruction at a time. In our characterization, C code compiled through the pipeline runs up to twice as slow as natively compiled code. The clearest losses come from hardware idioms. A CPU executes an idiom directly, yet the idiom typically reaches the kernel JIT as several instructions. We present BPF-EXT, a mechanism that extends the eBPF compilation pipeline via kernel modules with new operations, with minimal trusted computing base (TCB). Each operation in a BPF-EXT module carries two forms: a *proof sequence* of vanilla eBPF instructions for verifier checks and a *native emit* of machine instructions for JIT compilation. The native emit is the only per-operation addition to the TCB. With BPF-EXT, we build KINSN, including hardware-idiom operations such as rotate and conditional select. Lean 4 proofs can further discharge that trust, showing that each native emit computes the same result as its proof sequence. On x86-64 and ARM64, KINSN speeds up eBPF microbenchmarks by up to 24% and production applications by up to 12% over the unmodified pipeline. The same BPF-EXT mechanism supports whole-program native replacement, reaching 2.358× at the cost of a much larger TCB.

CCS Concepts: • **Software and its engineering** → **Just-in-time compilers**; *Formal language definitions*; *Compilers*; • **Computer systems organization** → *Embedded systems*.

Keywords: eBPF, JIT compilation, kernel extensions, instruction set extension, verified compilation, proof-carrying code

1 Introduction

Extended Berkeley Packet Filter (eBPF) safely extends OS kernels in domains including networking [5, 20], observability [3, 11], security [2, 6], and scheduling [7]. eBPF’s safety comes from a compilation pipeline that verifies every program before execution. A userspace compiler lowers each program to eBPF bytecode, which the in-kernel verifier checks for safety properties such as memory safety and termination. The kernel’s just-in-time compiler (the kernel JIT) then translates the verified bytecode to native code one opcode at a time. The kernel keeps the JIT simple to stay trustworthy.

The simple kernel JIT, however, is where eBPF loses performance. The same C code runs up to twice as slow through the pipeline as when compiled directly to native code (§3). An optimizing compiler fed the same verified bytecode recovers most of the gap. The gap therefore comes from the kernel JIT’s per-opcode code generation.

The clearest losses come from *hardware idioms*, operations that a CPU executes directly, usually as a single instruction. The eBPF instruction set has no single-instruction form for these operations. Optimization happens almost entirely in the userspace compiler. Even dedicated eBPF optimizers such as Merlin [19], K2 [30], and EPSO [33] rewrite programs within the instruction set. These idioms reach the kernel spelled out as multi-instruction sequences, and the per-opcode kernel JIT keeps the spelling. A 64-bit rotate with a variable shift, a single ROL instruction in native code on x86-64, reaches the kernel JIT as eight bytecode instructions and leaves as 15 machine instructions.

Our response is to let the simple kernel JIT emit these idioms directly, usually as a single native instruction. The idioms the kernel JIT compiles poorly form a small, bounded set, so a handful of additions suffices. One alternative, an optimizing compiler in the kernel, would enlarge the trusted computing base. The verifier checks only the bytecode, so the kernel would have to trust a compiler far larger than the per-opcode kernel JIT. Such a compiler would also grow the kernel code maintained for each architecture. Emitting the idioms is an instance of a more general problem, extending a verified compilation pipeline with new operations while keeping the trusted computing base small.

Any such extension must keep eBPF’s safety guarantee intact, with the existing verifier proving every loaded program safe. New operations, however, arrive from untrusted code outside the verifier and the JIT. The verifier and the JIT also carry years of analysis and formal verification [23, 27]. The kernel change for new operations must therefore stay small and one-time. The native code is specific to each architecture, while the verifier’s reasoning must stay architecture-independent.

We present BPF-EXT, a mechanism that extends the eBPF compilation pipeline with new operations. Our key idea is to give each operation two forms, a *proof sequence* of vanilla (unextended) eBPF that the existing verifier checks, and a *native emit* that the CPU runs. A compile-time recognizer rewrites programs to use the new operations. The verifier re-checks the proof sequence on every load. Neither the

recognizer nor the proof sequence joins the trusted computing base. A recognizer bug therefore cannot make a verified program unsafe.

With BPF-EXT, we build KINSN, a set of seven hardware-idiom operations, including rotate, conditional select, and bit extract. The verifier never sees the native emit that the CPU runs. The native emit is the only per-operation addition to the trusted computing base. We therefore prove in Lean 4 that, for each operation, the native emit has the same eBPF-visible effect as the proof sequence, carrying the verifier’s guarantee to the native code.

We implement BPF-EXT in the Linux kernel and in userspace tooling, as a small change to the kernel core, a compile-time recognizer, and kernel modules that supply the operations. KINSN speeds up eBPF microbenchmarks by up to 24% on x86-64 and 22% on ARM64 over the unmodified pipeline, recovering up to 42% of the gap to native code, and improves production application throughput by up to 12%. Native code size shrinks by 12–23% with no change to the existing verifier analysis (§7). The same BPF-EXT mechanism also supports whole-program native replacement, reaching 2.358× on Cilium at the cost of trusting the application’s native code (§7.4). A new operation ships as a kernel module, and the existing verifier analysis stays untouched.

Contributions. We make the following contributions:

- We present BPF-EXT, a mechanism that extends the eBPF compilation pipeline with new operations while reusing the existing verifier for safety (§4).
- With BPF-EXT, we build KINSN, a set of seven hardware-idiom operations that the simple kernel JIT emits directly (§5).
- We prove in Lean 4 that each KINSN native emit has the same eBPF-visible effect as the proof sequence the verifier checks, so the verifier’s guarantee carries over to the native code (§5).
- An implementation in the Linux kernel and userspace tooling (§6) recovers up to 42% of the eBPF-native gap in our evaluation (§7).

2 The eBPF Compilation Pipeline

This section describes the three stages of the eBPF compilation pipeline, with attention to how much each stage optimizes the program and to the instruction-set boundary that every stage respects. Every eBPF program reaches the kernel through the pipeline, shown in Figure 1. A user-space compiler produces bytecode, which the kernel verifies and JIT-compile. The compiled program then attaches to a hook such as XDP or tc.

Source compilation. The user-space compiler does most of the pipeline’s optimization. A language front end such as Clang lowers the source to LLVM IR, and LLVM’s middle end runs its full optimization passes. The eBPF backend

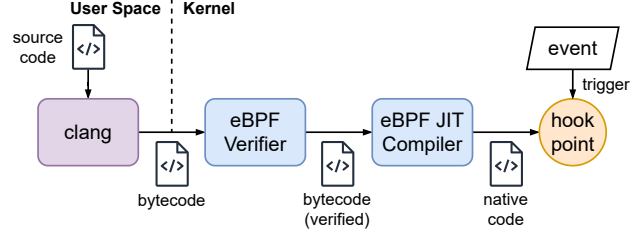


Figure 1. The eBPF execution pipeline.

then lowers the result to bytecode restricted to the operations the instruction set defines. Optimization research also targets this stage, where compile-time optimizers such as Merlin [19], K2 [30], and EPSO [33] rewrite programs for speed and size. The optimizers all stay within the instruction set, a boundary the verifier enforces. The instruction set has no single-instruction form for hardware idioms. Every optimizer at this stage, including LLVM, therefore spells the idioms out as multi-instruction sequences.

Verification. The verifier checks safety, changes the program little, and enforces the instruction set. The verifier proves memory safety, termination, and similar properties by static analysis [10]. The analysis proceeds opcode by opcode, over the 123 opcodes the instruction set defines [25]. An opcode the instruction set does not define is rejected. The rejection makes the instruction set a hard boundary for every upstream optimizer, so the idioms reach the JIT still spelled out as sequences. After a program passes, the verifier applies only a few targeted rewrites, such as dead-code elimination. A few systems also optimize at this load-time stage, such as KFuse [13], which merges verified programs within the instruction set. The verifier then hands the program to the JIT. The kernel also exposes *kfuncs*, native kernel functions that eBPF programs may call, registered by loadable modules with BTF type metadata [16]; BPF-EXT reuses this registration infrastructure (§6.1).

JIT compilation. The kernel’s per-opcode JIT does almost no optimization. The kernel maintains one such JIT for each supported architecture. Translation proceeds one opcode at a time, with the eBPF registers *r0*–*r10* mapped to fixed native registers. The only optimization choices are within one instruction’s encoding, such as picking the shortest immediate form on x86-64. The JIT runs no general cross-instruction optimization, such as global register allocation or instruction scheduling. The simple design keeps the JIT small and trustworthy beside the verifier, a tradeoff that §3.5 returns to. The multi-instruction sequences the backend emits therefore survive into the machine code unoptimized, at a cost that §3 measures.

3 Characterization of the eBPF-Native Performance Gap

The same source program runs $1.57\times$ slower on x86-64, and $1.98\times$ slower on ARM64, through the eBPF compilation pipeline than under direct native compilation (Table 1). We characterize the gap on pure-bytecode microbenchmarks, computation kernels that use no helper calls or map accesses, through three questions:

- **C-RQ1.** How much does in-kernel execution slow native code?
- **C-RQ2.** How much of the gap can an optimizing compiler recover from the same verified bytecode?
- **C-RQ3.** Which operations does the per-opcode kernel JIT compile poorly?

The three questions decompose the gap. The first separates the cost of in-kernel execution from the quality of the machine code the pipeline generates. Within code generation, the second weighs the per-opcode kernel JIT against the bytecode the JIT compiles. The third identifies which operations the JIT compiles poorly, so a fix can target the operations. The section closes with why the kernel cannot host an optimizing compiler (§3.5).

3.1 Experimental Setup

Benchmarks. The corpus is 27 pure-bytecode microbenchmarks. Each is a computation kernel from a real eBPF program in networking, tracing, or security. The examples include a consistent-hash load-balancer lookup from *katran* and a SipHash-like 64-bit-rotation mixer. The kernels use no helper calls or map accesses, so each measurement isolates instruction execution. The gap therefore characterizes the bytecode-to-native code-generation quality; in production programs, helper and map time dilutes the instruction-execution share, so case study in §7 measures the end-to-end effect separately.

Execution paths. We compile each benchmark through four execution paths, the kernel eBPF JIT baseline plus the three paths in Table 1. The paths vary the final backend and where the code runs. The kernel eBPF JIT compiles the verified bytecode in the kernel and is the baseline. In-kernel native and userspace native run native code from the same source, inside and outside the kernel. Userspace LLVM-BPF recompiles the same bytecode with an optimizing compiler in userspace [32]. Three pairwise comparisons attribute the gap. In-kernel native against userspace native isolates in-kernel execution. The kernel eBPF JIT against userspace LLVM-BPF isolates the per-opcode kernel JIT, given the in-kernel execution bound from C-RQ1. Userspace LLVM-BPF against userspace native isolates the bytecode form.

Metric. A path’s speedup on a benchmark is the kernel eBPF JIT’s median steady-state execution time divided by the path’s time. Each reported speedup is the geometric

Table 1. Speedup over the kernel eBPF JIT on the 27 pure-bytecode microbenchmarks, as the geometric mean of median steady-state time. A path counts as faster on a benchmark when it beats the kernel eBPF JIT by more than 2%.

Platform	Path	Speedup	# Cases Faster
x86-64	In-kernel native	$1.55\times$	24/27
	Userspace native	$1.57\times$	25/27
	Userspace LLVM-BPF	$1.53\times$	25/27
ARM64	In-kernel native	$1.88\times$	27/27
	Userspace native	$1.98\times$	27/27
	Userspace LLVM-BPF	$1.92\times$	27/27

mean over the benchmarks. Load, link, and compile time stay outside the speedup.

3.2 C-RQ1: How Much Does In-Kernel Execution Slow Native Code?

In the kernel, native code runs about as fast as in userspace. On x86-64, Figure 2 shows in-kernel native at $1.55\times$ and userspace native at $1.57\times$ over the kernel eBPF JIT, a difference of about 2%. On ARM64 the difference is about 5%, $1.88\times$ against $1.98\times$. The two paths run native code from the same source and differ in where the code runs. The spread therefore bounds the cost of in-kernel execution at about 2–5%. The cost is small, and the gap comes mainly from code generation.

Finding 1: *The eBPF-native gap comes from code generation. In-kernel execution slows native code by only about 2% on x86-64 and 5% on ARM64.*

3.3 C-RQ2: How Much Can an Optimizing Compiler Recover?

Code from an optimizing compiler on the same bytecode runs almost as fast as native code. Userspace LLVM-BPF compiles the same verified bytecode that the kernel runs, but with an optimizing backend in place of the per-opcode kernel JIT. As Table 1 shows, userspace LLVM-BPF reaches $1.53\times$ on x86-64 and $1.92\times$ on ARM64. Against userspace native at $1.57\times$ and $1.98\times$, the optimizing backend recovers 93% and 94% of the gap. Given the in-kernel execution bound from C-RQ1, the recovered share measures the performance the per-opcode kernel JIT loses. The remaining share of the gap, 7% on x86-64 and 6% on ARM64, comes from the bytecode form, which fixes choices such as the register set and calling convention before any compiler runs.

Finding 2: *The per-opcode kernel JIT is the dominant source of the generated code’s slowness. An optimizing compiler recovers 93–94% of the gap from the same verified*

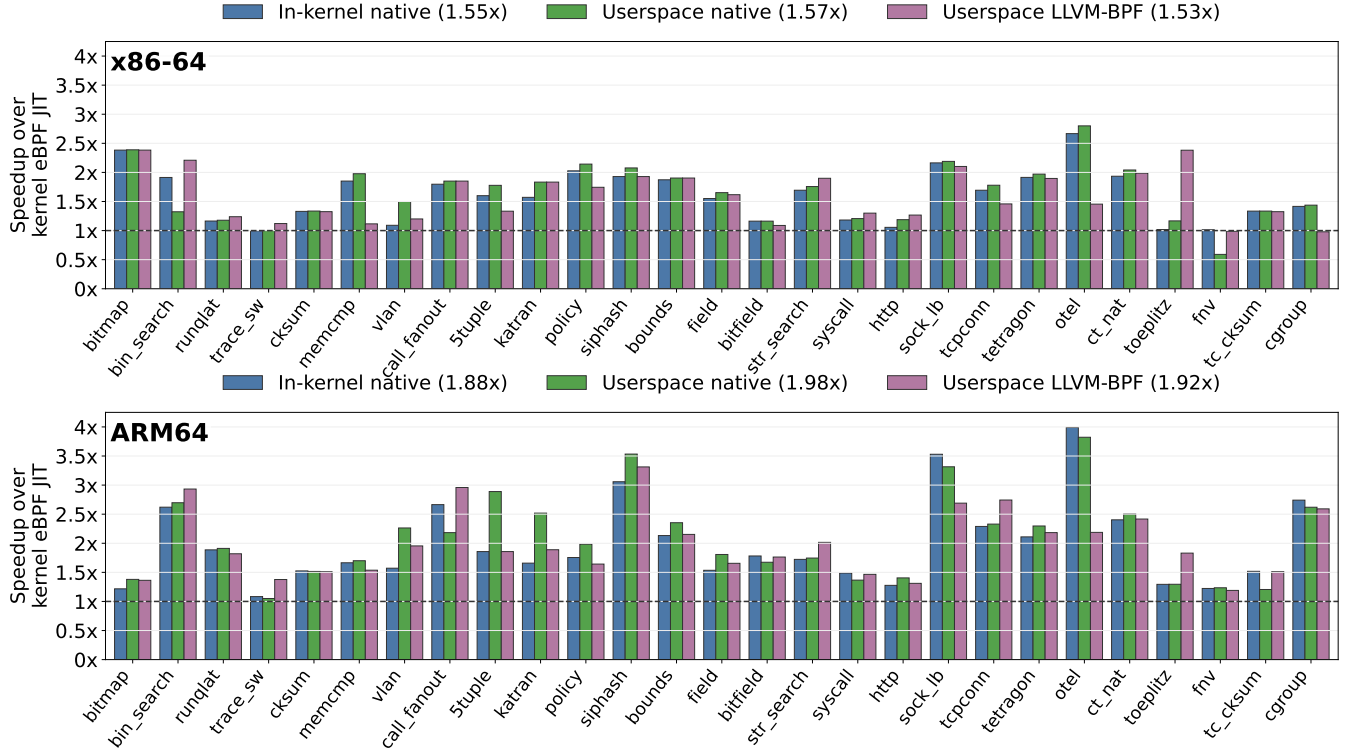


Figure 2. Per-case speedup over the kernel eBPF JIT on the 27 pure-bytecode microbenchmarks, on x86-64 (top) and ARM64 (bottom). Each bar is one non-baseline path. The ARM64 paths come from separate runs, each over its own baseline.

bytecode. The verified bytecode carries most of what a compiler needs to generate fast code.

3.4 C-RQ3: Which Operations Does the Kernel JIT Compile Poorly?

The per-opcode kernel JIT compiles hardware idioms poorly. A hardware idiom has no single-instruction form in the eBPF instruction set, so the eBPF backend spells the idiom out as several bytecode instructions (§2). The kernel JIT then translates each bytecode instruction separately into one or more machine instructions. Figure 3 shows a 64-bit rotate with a variable shift under the per-opcode kernel JIT and under direct native compilation. The rotate reaches the kernel JIT as eight bytecode instructions and leaves as 15 machine instructions. Direct native compilation uses a single rotate instruction, ROL on x86-64 or ROR on ARM64.

A small, bounded set of hardware idioms expands the same way under the kernel JIT. The rotate is one of a handful of idioms with no single-instruction form. Table 2 lists the idioms, with the instruction counts along each path. Across five idiom-stress probes on x86-64, small programs that each repeat one idiom, the kernel JIT emits 1.25–2.7× as many instructions as userspace LLVM-BPF for the same programs. Code size shows the cost across the whole corpus. In-kernel

native machine code is about half the size of the kernel JIT output, 0.54× on x86-64 and 0.49× on ARM64, an aggregate that includes losses beyond the idioms.

Finding 3: A significant share of the per-opcode JIT’s loss comes from a small, bounded set of hardware idioms, such as rotate, wide load, and conditional select. The remaining loss comes from cross-instruction optimizations, such as register allocation and instruction scheduling, that are outside the scope of a per-idiom fix.

3.5 Why Not an Optimizing Compiler in the Kernel?

The obvious response to the simple kernel JIT is to replace the JIT with an optimizing compiler. An optimizing compiler in the kernel would enlarge the trusted computing base behind eBPF’s safety guarantee. The verifier checks only the bytecode, so the kernel would have to trust the far larger compiler that produces the native code. Such a compiler would also grow the kernel code that must be maintained and audited for each supported architecture. A practical fix instead keeps the simple kernel JIT and targets the idiom share of the loss by letting the JIT emit a bounded set of hardware idioms directly.

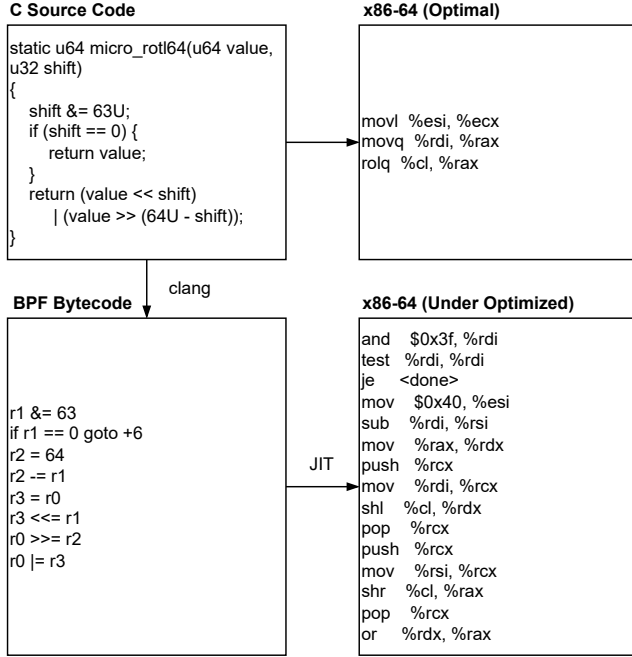


Figure 3. Compiling a 64-bit rotate with a variable shift. Direct compilation to x86-64 emits a single ROL. Through eBPF the rotate becomes eight bytecode instructions, which the per-opcode kernel JIT translates separately into 15 machine instructions (Linux 6.1, x86-64).

Table 2. Hardware idioms and their instruction counts along each path, measured on x86-64 from real JIT dumps and compiler output. The second column names the native instruction on x86-64 / ARM64. Byte swap assumes a MOVBE-capable x86-64. The bulk-copy row is a 64-byte copy between non-overlapping regions.

Operation	Native instruction	# Instructions		
		Native	eBPF	Kernel JIT
Rotate	ROL/ROR	1	8	15
Wide load	MOV/LDR	1	22	22
Conditional select	CMP+CMOV/CMP+CSEL	2	3	4
Byte swap	MOVBE/REV	1	2	2
Bulk copy (64 B)	MOVUPS/LDP+STP	4	16	16

Emitting the idioms from the simple kernel JIT must meet three constraints, which the next section addresses. First, the verifier must still prove every program safe, which keeps eBPF’s existing guarantee unchanged. Second, adding an operation must not require rewriting the verifier or the JIT core, which keeps the kernel change small and one-time. Third, one operation must map to a different native instruction on each architecture, which keeps verification of the operation architecture-independent. Within these constraints the fix

targets the operations C-RQ3 identifies. The in-kernel execution cost (C-RQ1) and the bytecode-form residual (C-RQ2) stay out of scope, and §7 measures how much of the JIT’s loss the idioms carry.

4 BPF-EXT: Extending the eBPF Compilation Pipeline

BPF-EXT extends the eBPF compilation pipeline with new operations that the existing verifier checks and the kernel JIT emits directly. Each operation carries two forms, a *proof sequence* of vanilla eBPF that the verifier checks and a *native emit* that the CPU runs. Figure 4 contrasts the pipeline with and without BPF-EXT. A compile-time recognizer rewrites an operation’s pattern into one or more *extended instructions*, each a pair of a call and a sidecar that carries the operands. Each call selects a *descriptor* from a kernel module, and the descriptor supplies both forms for its instruction. The same program therefore verifies as vanilla eBPF and runs as native code. BPF-EXT meets the three constraints of §3.5: the safety guarantee stays intact up to one explicit obligation (§4.3), the kernel core changes once (§4.2), and verification stays architecture-independent (§4.1). §5 then presents KINSN, the hardware-idiom operations built with the mechanism.

4.1 The Compilation Pipeline

Recognition. The recognizer matches the patterns of supported operations in userspace, before the program loads (§6.2). For KINSN’s rotate, the pattern is the shift-and-or sequence of Figure 3. The call and the sidecar reuse existing eBPF opcodes. BPF-EXT therefore adds no new opcode.

Verification. The verifier checks an extended instruction by lowering it to vanilla eBPF. The proof sequence is a fragment of vanilla eBPF with the same semantics as the extended instruction. A lowering stage substitutes the proof sequence for the extended instruction before the main analysis. The verifier then analyzes the lowered program with its existing transfer functions. After the analysis succeeds, a restore stage reinstates the extended instructions for the JIT.

Execution. The JIT dispatches each extended instruction to the descriptor’s native emit for the running architecture. For KINSN’s rotate, the native emit is a single ROL on x86-64 or a single ROR on ARM64. The exact proof sequence, like the native emit, can differ across architectures. Every proof-sequence variant stays vanilla eBPF. Verification therefore stays architecture-independent. When a descriptor provides no native emit for the architecture, the extended instruction falls back to its proof sequence, which the JIT compiles as vanilla eBPF. The interpreter has no dispatch for extended instructions. A kernel without a JIT therefore rejects programs that carry extended instructions.

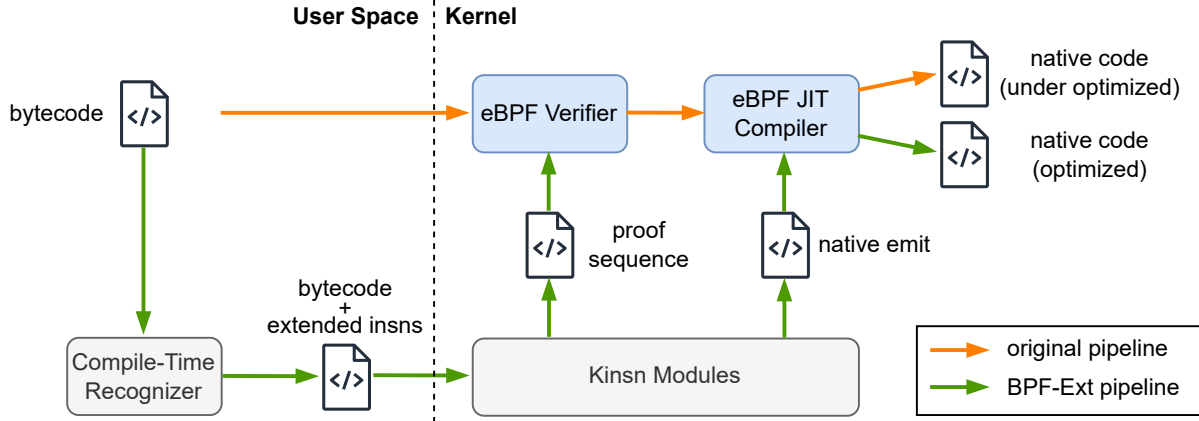


Figure 4. The original eBPF pipeline (orange) and the BPF-Ext pipeline (green). The original pipeline verifies and JIT-compiles the bytecode unchanged, producing under-optimized native code. With BPF-Ext, the compile-time recognizer rewrites supported operations into extended instructions, and the KINSN modules supply each operation’s proof sequence to the verifier and its native emit to the JIT, producing optimized native code.

4.2 Adding an Operation

A new operation needs only descriptors in a kernel module and a pattern in the recognizer. A descriptor is a small kernel structure with two author-supplied parts, the proof-sequence routine and the per-architecture native emits. A family of descriptors implements an operation, one descriptor per variant, such as width, condition, or architecture. A kernel module registers its descriptors with the kernel core. BPF-Ext’s one-time change to the kernel core stays the same for every operation (§6).

The mechanism leaves an operation’s size open. A proof sequence may span a few instructions or an entire region within the rules of §4.3.

4.3 Safety and the Trusted Base

BPF-Ext does not add operation-specific abstract semantics to the verifier. Everything the recognizer and the module contribute reaches the verifier inside the lowered program. The verifier re-checks the lowered program on every load, with the same analysis as for vanilla eBPF. A faulty rewrite or proof sequence that violates safety therefore fails verification like any unsafe program.

The verifier’s verdict on the lowered program carries over to the restored program. The restore stage collapses each *proof region*, the span its proof sequence occupies, back into its extended instruction and changes nothing else. For the collapse to preserve the verdict, a region must stay single-entry and single-exit, free of nested extended instructions, calls, exits, and backward jumps. Control then enters a region only at its top and leaves only past its end, like a single instruction. The kernel validates proof-region well-formedness at load time, including single entry, single exit, no nested extended instructions, and no calls or exits inside a region. The kernel

also validates that no jump enters or leaves a region except at its boundary.

The trusted enforcement path therefore consists of the existing eBPF verifier and JIT plus a small, generic BPF-Ext patch that validates proof regions, lowers extended instructions before verification, restores them after verification, and dispatches their native emits (§6). The per-operation native emit is the one component whose correctness BPF-Ext cannot check at load time. Every operation must therefore supply evidence that its native emit has the same eBPF-visible effect as its proof sequence. For KINSN, the Lean 4 proofs of §5 supply the evidence. The userspace recognizer is outside the safety boundary: an incorrect rewrite produces a program the verifier re-checks and rejects or accepts on its own merits.

5 KINSN: Hardware-Idiom Operations

With BPF-Ext, we build KINSN, a set of seven operations for hardware idioms. Each operation targets an idiom for which the eBPF instruction set has no single-instruction form. The rotate, for example, covers its 64-bit and 32-bit variants through separate descriptors. The section closes with the Lean 4 proofs that supply the evidence §4.3 requires.

The seven operations are a deliberate choice within the space that §4.2 leaves open. The idioms of the kind §3.4 characterizes are few. A handful of modules therefore covers the kernel JIT’s clearest losses. Each proof sequence stays a small fragment of the program. The equivalence evidence each operation owes covers only that fragment. The compiler spells six of the seven idioms out as fixed bytecode patterns, which the recognizer matches in place. The prefetch alone has no eBPF form of any length. The recognizer instead

Table 3. The seven KINSN operations, each with the canonical proof sequence and the native emit per architecture. Exact proof sequences vary by descriptor and architecture (§4). A dash marks an architecture with no native emit, where the operation falls back to its proof sequence. Each native-emit cell lists the instructions emitted for one operation instance. Table 2 counts the instructions along each compilation path for the idioms the two tables share, and the native compiler’s instruction choice there can differ from the native emit here.

Operation	Proof sequence	Native emit	
		x86-64	ARM64
Rotate	two shifts + or	ROL	ROR
Conditional select	branch + move	CMP+CMOV	TST+CSEL
Bit extract	shift + and	BEXTR	UBFX
Byte swap	load + byte swap	MOVBE	LDR+REV
Prefetch	no-op (JA +0)	PREFETCHT0	PRFM
Bulk copy	loads + stores	MOV	LDP+STP
LEA	shift + adds	LEA	—

inserts the prefetch at sites it selects, with a no-op proof sequence.

Table 3 lists the seven operations, each with its proof sequence and its native emit on x86-64 and ARM64. The native emits differ across architectures, in shape and in availability. A conditional select compiles to one extended instruction on x86-64, emitting the CMP+CMOV pair, and to two extended instructions on ARM64, one emitting TST and one emitting CSEL. The byte swap, fused with its load, splits the same way on ARM64, into an LDR and a REV. One operation, the address computation LEA, exists on x86-64 alone.

Formal verification. The dual lowering of a KINSN: one path expands to vanilla eBPF for the verifier, the other emits native code in the JIT, imposes a proof obligation. The native sequence that the CPU actually executes must produce the same eBPF-observable architectural state as the eBPF expansion. We therefore prove, in Lean 4, that for each supported KINSN operation, the eBPF expansion and the corresponding native sequence produce the same eBPF-observable architectural state. For register-only operations such as rotate, conditional select, and bit extract, the refinement reduces to equality on the destination register. For memory-touching operations such as byte swap (fused with its load) and bulk copy, the proof obligation is equality on the projected architectural state over live registers and the accessed memory, leaving all other eBPF-visible memory unchanged. Prefetch is modeled as an architectural no-op; microarchitectural effects such as cache state are outside the semantic model. We give each ISA a small-step register-transfer semantics; a program denotes the fold of its per-instruction transitions. The native instruction’s semantics is ported from the existing formalization [1]. The cross-ISA proof is factored through an intermediate hardware-independent specification. The

Table 4. The kernel core is a 929-line addition, measured as lines added on the BPF-EXT kernel branch over mainline Linux. The total excludes 11 lines of disassembler support and tools-side header sync.

Component	Lines of code
Verifier (lowering, restore, validation)	487
BTF and registration	139
JIT dispatch (ARM64)	121
JIT dispatch (x86-64)	97
Headers	85
Total kernel core	929

refinement is stated for verifier-accepted proof sequences and verifier-safe initial states. For each KINSN operation, we define a specification over the relevant architectural state and discharge ISA-local correctness lemmas stating that the eBPF expansion and the native sequence each refine the specification.

6 Implementation

We implement BPF-EXT in three parts. The kernel core is a one-time change to the verifier and the JIT (§6.1). A compile-time recognizer rewrites idioms in userspace before load (§6.2). Each operation ships as one or more loadable kernel modules (§6.3).

6.1 Kernel Core

The kernel core adds 929 lines to the Linux kernel. Table 4 breaks the total down by component.

Instruction encoding. Each extended instruction is a pair of instructions, a call and a sidecar, shown in Table 5. Both reuse existing eBPF opcodes, and a reserved `src_reg` value marks the pair for BPF-EXT. The call identifies the descriptor to use. The sidecar carries a packed 52-bit payload with the operands. The low four bits of the payload must form a valid eBPF register number. The recognizer therefore re-encodes a payload whose low four bits exceed the largest register number. The kernel reverses the re-encoding when reading the sidecar.

Verifier lowering and restore. The verifier gains a lowering stage and a restore stage around its existing analysis. The lowering stage walks the program backward and replaces each extended instruction with the proof sequence from its descriptor. The stage records each original pair for restoration. The verifier then runs unchanged on the lowered program. The restore stage reinstates the extended instructions after verification succeeds. With proof-sequence validation and the supporting bookkeeping, the changes to the verifier total 487 lines, the largest single component.

Table 5. Extended-instruction encoding. A reserved `src_reg` value marks the pair for BPF-EXT, a BPF_CALL for the call and a MOV64_K for the sidecar. The notation `[h:l]` gives a field’s position within the 52-bit sidecar payload.

Insn	Field	Bits	Meaning
Call	imm	32	BTF ID of the operation’s stub kfunc
	off	16	Index into <code>fd_array</code> for the module’s BTF file descriptor
Sidecar	dst_reg	[3:0]	Destination register or per-descriptor form tag
	off	[19:4]	Offset or auxiliary field per descriptor
	imm	[51:20]	Immediate value or configuration per descriptor

JIT dispatch. The x86-64 and ARM64 JIT backends each gain a dispatch case for extended instructions. On reaching an extended instruction, the JIT looks up the descriptor and decodes the sidecar payload. The JIT then calls the descriptor’s native emit for the running architecture. When a descriptor has no native emit for the architecture, the kernel rewrites the site back to its proof sequence during load-time fixups, before the JIT runs. The dispatch logic is 97 lines on x86-64 and 121 lines on ARM64.

Module registration. A loaded module registers its descriptors for lookup by the verifier and the JIT. Registration reuses two existing kernel facilities, BTF, the kernel’s type metadata for eBPF objects, and kfuncs, kernel functions that eBPF programs may call. Each module exports a stub kfunc as its BTF anchor. The module lists its `struct bpf_kinsn` descriptors in a parallel array. The kernel caches the descriptors per program. The BTF and registration changes are 139 lines. The new `struct bpf_kinsn` type and helpers add 85 lines of headers.

6.2 Compile-Time Recognizer

The recognizer rewrites a program’s idioms into extended instructions before load, through two paths. A program built without the recognizer, or one whose idioms the recognizer does not match, produces the same vanilla eBPF bytecode and runs through the unchanged pipeline.

Selector. The first path is a selector in the LLVM eBPF backend. The selector matches idioms during instruction selection and emits the call and sidecar in place of the decomposed bytecode. Counted over the backend’s BPF-EXT commits, the selector adds 2,126 lines, half in a new selection module.

Bytecode recovery. The second path is a bytecode-recovery pass that re-matches idioms on already-compiled bytecode. The pass covers bytecode the selector never sees or cannot match, such as programs compiled by a separate toolchain. The implementation is a standalone userspace tool of 6,967

lines. The pass checks that each rewrite preserves the program’s semantics, for example that a temporary register the idiom overwrites is dead afterward. Even a wrong rewrite leaves the program safe, because the verifier re-checks every lowered program (§4.3).

6.3 Operation Modules

Each operation is packaged as one or more kernel modules, separate from the kernel core. A module supplies an operation’s proof-sequence routine, its native emits, and the BTF registration that exposes both.

Each module covers one native instruction on one architecture. An operation that targets a single architecture contributes modules on that architecture only, which is why the per-architecture counts differ. The module tree holds 14 modules on x86-64 and 11 on ARM64, 9,765 lines in all, including shared helper headers, test-only modules, and operations beyond KINSN’s seven (Table 3).

7 Evaluation

This section evaluates KINSN, the hardware-idiom instantiation of BPF-EXT. BPF-EXT provides the extension mechanism, while KINSN provides the concrete descriptor families used in our experiments. KINSN operates below the instruction-set boundary, complementary to bytecode optimizers [19, 30, 33] that cannot emit native idioms the ISA lacks. We use two workload classes. First, we reuse the 27 pure-bytecode microbenchmarks from §3 to isolate instruction-selection effects. Second, we use production datapath applications to test whether those effects matter in real BPF loading and execution paths. We answer four questions:

- **RQ1:** How effectively does KINSN improve generated native-code efficiency across architectures?
- **RQ2:** How does KINSN perform on production applications?
- **RQ3:** How sensitive is KINSN to family-selection policy?
- **RQ4:** How does BPF-EXT compare with a native-in-kernel upper bound?

Evaluation environment. We use two evaluation platforms. The x86-64 experiments run in a KVM virtual machine with 8 vCPUs and 64 GB RAM on an Intel Xeon Silver 4210R @ 2.40 GHz host, using Linux 7.0.0-rc2+. The ARM64 experiments run on an AWS t4g.small instance (2 vCPUs, 2 GiB RAM, AWS Graviton2), using a KINSN-enabled Linux 7.0.0-rc2+ build. These platforms cover both the microbenchmark suite and the production-application experiments. For matched application comparisons, the stock and KINSN runs use the same kernel image and differ only in the loaded program form or KINSN policy.

Measurement conventions. Unless otherwise stated, each experiment uses three measured runs and reports the median. Execution-time and throughput ratios are normalized to the stock kernel eBPF JIT, so values above 1× indicate faster execution or higher throughput. BPF-cost ratios report post/baseline cost, so values below 1× indicate lower BPF runtime cost. For BPF-counter results, we report post/baseline ns/run for retained counter rows. For each retained row, we compute $\text{run_time_ns_delta}/\text{run_cnt_delta}$, drop rows with fewer than 100 runs in either phase, and pair rows by program name, program type, and occurrence index.

7.1 RQ1: x86-64 and ARM64 Microbenchmark Signal

KINSN reduces generated native-code size on both evaluated architectures. Relative to the stock kernel eBPF JIT, generated kernel JIT code shrinks to 0.772× on x86-64 KVM and 0.879× on ARM64 AWS, corresponding to geomean reductions of 22.8% and 12.1%, respectively.

Figure 5 reports the corresponding execution-time results. On x86-64 KVM, the KINSN-enabled run achieves a 1.242× geomean speedup over the stock kernel eBPF JIT, reducing execution time by 19.47%. On ARM64 AWS, the KINSN-enabled run applies all matched KINSN sites, with 308/308 sites in the median sample and 924/924 site instances across all samples. Over the 27 KINSN-bearing benchmarks, it achieves a 1.222× geomean speedup over the stock kernel eBPF JIT, reducing execution time by 18.17%. Both architectures produce zero correctness mismatches. On the microbenchmarks, the 1.242× speedup recovers 42% of the characterization’s 1.57× eBPF-native gap on x86-64 $((1.242-1)/(1.57-1))$.

The per-benchmark distributions suggest different sources of benefit across architectures. On x86-64, speedups are more closely associated with native code-size reduction, which is consistent with KINSN replacing conservative stock-JIT instruction sequences and reducing dynamic instruction overhead. On ARM64, code-size reduction alone is less explanatory: the largest gains appear when the selector reaches target-specific idioms such as rotate/extract operations and packet-field load fusion. For example, `siphash_rotate64_mixer` reaches a 1.902× speedup on ARM64 with dominant extract-style sites, while networking-oriented microbenchmarks such as Cilium socket load balancing, BCC TCP tuple filtering, and Cilium CT/NAT tuple rewriting achieve speedups of 1.691×, 1.624×, and 1.482×, respectively. These results suggest that KINSN is not merely a code-size optimization. Rather, it acts as an architecture-sensitive specialization mechanism whose effectiveness depends on the stock JIT baseline, the selected descriptor family, and whether the selected sites lie on performance-critical execution paths.

The regressions also reveal the current boundary of the selector. `bpftrace_comm_key_fnv_hash` regresses to 0.862× on ARM64, while `bi_tmap_popcount_scan` regresses to 0.966× despite having applied KINSN sites. These cases indicate that

KINSN is not uniformly beneficial and motivate descriptor-level gating or lightweight cost-model guidance, rather than blindly applying every matched KINSN site.

Load-time overhead. The KINSN lowering and restore stages add two linear passes over the program during verification. Across all 62 microbenchmarks on x86-64 KVM, the kernel-side load time (`object_load_ns`, covering verification and JIT) shows a 0.99× geomean ratio relative to the stock kernel, i.e., no measurable overhead. For programs with KINSN sites, the end-to-end compile time (including the userspace recognizer) increases by 1.4–2.4×, but remains sub-millisecond in absolute terms.

7.2 RQ2: Real Packet-Processing Applications

Microbenchmarks isolate individual instruction patterns, but packet-processing applications combine those patterns with maps, helper calls, tail calls, and application-specific datapath structure. We therefore evaluate KINSN on two real eBPF packet-processing applications: Cilium and Katran. Cilium represents a large tail-call-heavy Kubernetes datapath, while Katran represents a compact high-rate XDP load balancer with a hot directly attached program. We measure end-to-end datapath throughput with BPF runtime statistics disabled to avoid measurement overhead.

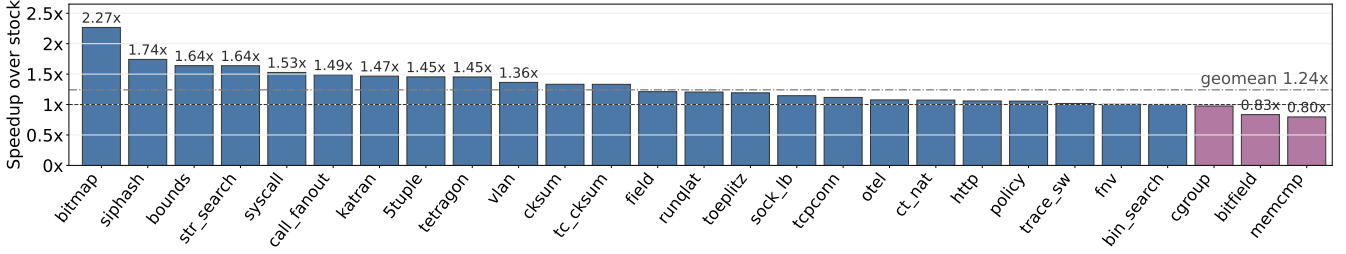
Cilium on x86-64. On x86-64 KVM, the default full-KINSN policy improves Cilium’s end-to-end datapath throughput by 1.074× over the stock kernel eBPF JIT. This run applies 4086 KINSN sites with zero load-time skips or report errors. The applied sites span several families: LEA contributes 2346 sites, conditional select 385, endian fusion 766, extract 2, and bulk memory 587. This confirms that KINSN reaches real bytecode patterns in Cilium’s production datapath.

Katran on ARM64. On ARM64 AWS, the conservative KINSN policy applies 21 sites and improves Katran’s end-to-end workload throughput by 1.073× over the stock kernel eBPF JIT. Because Katran’s datapath is dominated by a single hot XDP program, the throughput signal is clean: the program that receives KINSN sites is the program on the critical path.

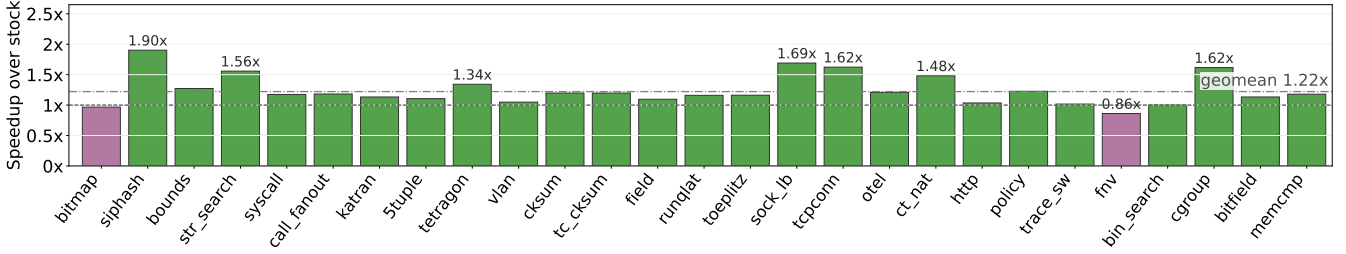
Together, these two case studies show that KINSN improves real packet-processing workloads on both architectures. The results raise a follow-up question: once KINSN reaches real application bytecode, should it enable every matched family, or should it select families by workload-specific profitability?

7.3 RQ3: Family-Selection Policy Sensitivity

KINSN creates a policy surface above the BPF-EXT kernel mechanism. The kernel mechanism exposes a bounded set of verified descriptor families, but enabling a family is a performance decision: each matched site trades the dynamic work saved by a shorter native idiom against the cost and placement of the replacement in a particular datapath. A simple policy would maximize coverage, i.e., enable every



(a) x86-64 KVM KINSN-enabled run.



(b) ARM64 AWS KINSN-enabled run.

Figure 5. Microbenchmark execution-time speedup over the stock kernel eBPF JIT. Each benchmark is measured with three samples and INNER_REPEAT=100000; bars report speedup computed from median exec_ns, and higher is better. Panel (a) reports the x86-64 KVM KINSN-enabled run. Panel (b) reports the ARM64 AWS KINSN-enabled run over the 27 benchmarks with applied KINSN sites. Red lines mark the geomean speedup reported in each panel.

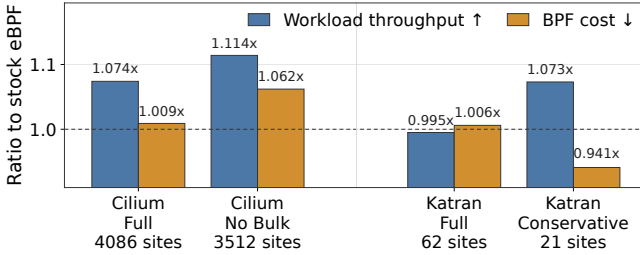


Figure 6. Application-level KINSN policy probes. Workload throughput ratios are higher-is-better, while BPF cost ratios are lower-is-better. Cilium compares the full KINSN policy with the no-bulk policy; Katran compares a more-sites policy with the selected-family policy. The dashed line marks the stock eBPF baseline.

implemented family and apply as many matched sites as possible. Figure 6 illustrates this policy effect by comparing two paired policy perturbations on Cilium and Katran.

For Cilium, we start from the full x86-64 KINSN policy used in RQ2. As shown in Figure 6, this policy applies 4086 sites and improves workload throughput to 1.074 $\times$, while the BPF-cost ratio remains near neutral at 1.009 $\times$. We then disable only the bulk_memory family while keeping the other KINSN families enabled. This reduces the number of applied sites from 4086 to 3512, but increases workload throughput further to 1.114 $\times$. At the same time, the BPF-cost ratio rises

to 1.062 $\times$, so this result should not be read as saying that bulk_memory is universally harmful. Rather, it shows that an additional family can increase syntactic coverage without improving the measured datapath: the saved instructions may not outweigh the replacement overhead, or they may not lie on the part of the datapath that dominates throughput.

Katran shows the same policy lesson more directly. Figure 6 shows that the conservative ARM64 policy used in RQ2 applies 21 sites and improves Katran to 1.073 $\times$ workload throughput, while reducing BPF cost to 0.941 $\times$. To test whether simply increasing coverage is beneficial, we then force a coverage-max policy that enables every implemented ARM64 family and applies 62 sites. The result is worse despite the larger number of applied sites: workload throughput falls to 0.995 $\times$, and BPF cost rises to 1.006 $\times$. Because Katran is dominated by a single hot XDP path, this result gives a particularly clear counterexample to coverage maximization: adding more matched sites does not translate into either higher throughput or lower BPF cost.

Taken together, the figure rejects raw site count as the right objective for KINSN policy. The useful objective is profitability: a descriptor family should be enabled when its replacements save enough dynamic work on the target workload to justify their cost, rather than simply because they increase the number of matched sites.

7.4 RQ4: Native-in-Kernel Upper Bound

The three RQs above evaluate KINSN as an instantiation of BPF-EXT. We now place BPF-EXT in a broader design space by using BPF-EXT itself to load whole-program native replacements. In this experiment, each Cilium BPF program is compiled from the same C source along two paths: the proof sequence is the normal eBPF bytecode that the verifier checks, while the native emit is the same source compiled with LLVM -O2 directly to native code. The verifier still checks the eBPF proof sequence on every load, but the equivalence between the two compilations is trusted rather than proved, so the application’s native code joins the TCB. The $2.358\times$ upper bound is not comparable to the characterization’s $1.57\times$ microbenchmark gap, which measures pure-bytecode kernels in isolation rather than Cilium’s full datapath.

We use Cilium for this upper-bound study because it is the same kind of datapath workload evaluated above. The native-in-kernel experiment uses the same x86-64 KVM application setup as the Cilium KINSN runs. It is not a claim about Cilium control-plane throughput: it measures steady-state datapath throughput after endpoint setup, with runtime reload and regeneration controllers disabled and the Cilium agent paused during measured workload samples.

Relaxing the trust boundary exposes a much higher performance ceiling. Cilium throughput improves by $2.358\times$ over stock eBPF, and retained BPF cost drops from 488.7 ns/run to 262.3 ns/run, a $1.86\times$ BPF-counter speedup.

The performance gain comes with a larger trusted surface. The Cilium native run logs 113 native replacements and 22 manifest no-match pass-throughs. The native sidecar data also records 89 Cilium manifest objects across 8 native files. These numbers are not a full TCB LOC audit, but they show the scale of the code surface that the native path must trust or validate.

This places stock eBPF, BPF-EXT instantiated as KINSN, and native-in-kernel execution at three points in the same design space. Stock eBPF is the most conservative point: it keeps the existing verifier/JIT trust boundary but pays the cost of a small virtual instruction set. Native-in-kernel execution is the performance ceiling under an assumed-safe model, but it trusts whole native replacements and the loader path. KINSN sits between them: its $1.074\times$ Cilium throughput gain recovers 5.4% of the $2.358\times$ native upper bound’s gap ($((1.074-1)/(2.358-1))$), compared with 42% on the microbenchmarks, because real datapaths spend most of their time in helpers, maps, and tail calls that KINSN does not target. KINSN does not reach the native upper bound, but it provides controllable speedups through a bounded, kernel-defined instruction surface rather than trusting whole native replacement objects.

8 Related Work

eBPF bytecode optimization. Bytecode optimizers improve eBPF programs within the existing instruction set. K2 [30] searches for shorter bytecode with stochastic synthesis. Merlin [19] rewrites programs at the LLVM-IR and bytecode levels. EPSO [33] applies rewrite rules from offline enumerative superoptimization. ePass [29] lifts bytecode into SSA form for program transformation. KFuse [13] merges chains of eBPF programs into one program after verification. BPF-EXT grows the set of operations available to a program.

Extending eBPF’s operations. The existing routes to new operations are kfuncs and instruction-set extensions. A kfunc exposes a kernel function for eBPF programs to call [16]. The verifier checks a kfunc call’s arguments against BTF types, and the body runs as native code outside the verifier’s analysis. hXDP [4] extends the eBPF instruction set for FPGA targets, and the translated programs run outside the verifier’s coverage. The eBPF instruction set is now an IETF standard [25], so a revision is a standardization effort. A BPF-EXT operation instead shows the verifier its full semantics, as a proof sequence in the lowered program (§4.1). A new operation ships as a kernel module plus a recognizer pattern, with no instruction-set revision (§4.2).

Relocating execution or the safety argument. Another line of work moves where eBPF runs or what its safety rests on. bpftime [32] runs eBPF programs in userspace. eBPF for Windows [21] checks programs in userspace with the PREVAIL verifier [10] and loads checked programs as signed driver modules. Craun et al. [8] verify, compile, and sign programs on a separate host for embedded devices. KFlex [9] moves only the extension’s own memory safety and termination to runtime checks. Rex [12] drops the verifier and accepts extensions written in safe Rust. MOAT [18] and SafeBPF [15] add runtime isolation beneath the verifier’s static checks. With BPF-EXT, programs stay in the kernel, under the existing verifier’s load-time check. Beyond BPF-EXT’s generic enforcement core, the only per-operation addition to the trusted base is the native emit (§4.3).

Verified compilation and proof-carrying code. Verified compilation provides the techniques behind the KINSN proofs. CompCert [14] established mechanized semantic preservation for a realistic compiler. Alive2 [17] checks refinement for each LLVM transformation. Jitk [28] compiles classic BPF through a Coq-verified JIT that replaces the seccomp interpreter path. Jitterbug [23] proves behavioral equivalence for per-instruction eBPF JIT translation with an SMT solver, and JitSynth [26] synthesizes such verified JITs from interpreter semantics. CertrBPF [31] derives a verified C implementation of an eBPF interpreter from a Coq specification. All of these systems target a fixed instruction set. Each KINSN operation instead carries its own Lean 4 proof that the native emit has the same eBPF-visible effect as its proof sequence (§5).

Beyond the translators, machine-checked proofs also target eBPF's verification and loading steps. Agni [27] verifies offline the eBPF verifier's range analysis, on whose soundness BPF-Ext's safety argument rests. BCF [24] supplies abstraction-refinement proofs from userspace, which the kernel checks at load time. KINSN's equivalence proofs stay offline, as development-time evidence that never enters the kernel. Proof-carrying code (PCC) [22] is the closest ancestor of BPF-Ext. Both admit untrusted code into the kernel only after an in-kernel check. PCC installs a new proof checker in the kernel. BPF-Ext reuses the existing eBPF verifier as the checker.

9 Conclusion

We presented BPF-Ext, a mechanism that extends the eBPF compilation pipeline with new operations the kernel JIT emits directly. Each operation carries a proof sequence that the existing verifier checks on every load and a native emit that the CPU runs. Beyond the generic enforcement core, the native emit is the only per-operation addition to the trusted base. With BPF-Ext, we built KINSN, a set of seven hardware-idiom operations. Lean 4 proofs show that each KINSN native emit has the same eBPF-visible effect as its proof sequence. KINSN speeds up eBPF microbenchmarks by up to 24% on x86-64 and 22% on ARM64 over the unmodified pipeline, recovering up to 42% of the gap to native code, and improves production application throughput by up to 12%. A new operation now ships as a kernel module, and the existing verifier analysis stays untouched.

References

- [1] Amazon Web Services. 2024. LNSym: Armv8 Native Code Symbolic Simulator in Lean. <https://github.com/leanprover/LNSym>. Accessed: 2026-06-09.
- [2] Aqua Security. 2026. Tracee: Runtime Security and Forensics using eBPF. <https://github.com/aquasecurity/tracee>. Accessed: 2026-04-06.
- [3] bpftrace Authors. 2019. bpftrace: High-level Tracing Language for Linux. <https://github.com/bpftrace/bpftrace>. Accessed: 2026-06-09.
- [4] Marco Spaziani Brunella, Giacomo Belocchi, Marco Bonola, Salvatore Pontarelli, Giuseppe Siracusano, Giuseppe Bianchi, Aniello Cammarano, Alessandro Palumbo, Luca Petrucci, and Roberto Bifulco. 2020. hXDP: Efficient Software Packet Processing on FPGA NICs. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*. USENIX Association, Virtual Event, 973–990. <https://www.usenix.org/conference/osdi20/presentation/brunella>
- [5] Cilium Authors. 2024. Cilium: eBPF-based Networking, Observability, and Security. <https://cilium.io>. Accessed: 2026-04-06.
- [6] Jonathan Corbet. 2019. KRSI—the other BPF security module. LWN.net, <https://lwn.net/Articles/808048/>. Accessed: 2026-04-06.
- [7] Jonathan Corbet. 2023. The extensible scheduler class. LWN.net, <https://lwn.net/Articles/922405/>. Accessed: 2026-04-06.
- [8] Milo Craun, Adam Oswald, and Dan Williams. 2023. Enabling eBPF on Embedded Systems Through Decoupled Verification. In *Proceedings of the 1st Workshop on EBPF and Kernel Extensions* (New York, NY, USA) (eBPF '23). Association for Computing Machinery, New York, NY, USA, 63–69. doi:10.1145/3609021.3609299
- [9] Kumar Kartikeya Dwivedi, Rishabh Iyer, and Sanidhya Kashyap. 2024. Fast, Flexible, and Practical Kernel Extensions. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 249–264. doi:10.1145/3694715.3695950
- [10] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzy, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and precise static analysis of untrusted Linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (PLDI 2019). Association for Computing Machinery, New York, NY, USA, 1069–1084. doi:10.1145/3314221.3314590
- [11] IO Visor Project. 2015. BCC: Toolkit for BPF-based Linux Tracing and Performance Analysis. <https://github.com/iovisor/bcc>. Accessed: 2026-06-09.
- [12] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 325–342. <https://www.usenix.org/conference/atc25/presentation/jia>
- [13] Hsuan-Chi Kuo, Kai-Hsun Chen, Yicheng Lu, Dan Williams, Sibin Mohan, and Tianyin Xu. 2022. Verified programs can party: optimizing kernel extensions via post-verification merging. In *Proceedings of the Seventeenth European Conference on Computer Systems* (Rennes, France) (EuroSys '22). Association for Computing Machinery, New York, NY, USA, 283–299. doi:10.1145/3492321.3519562
- [14] Xavier Leroy. 2009. Formal verification of a realistic compiler. *Commun. ACM* 52, 7 (July 2009), 107–115. doi:10.1145/1538788.1538814
- [15] Soo Yee Lim, Tanya Prasad, Xueyuan Han, and Thomas Pasquier. 2024. SafeBPF: Hardware-assisted Defense-in-depth for eBPF Kernel Extensions. In *Proceedings of the 2024 on Cloud Computing Security Workshop* (Salt Lake City, UT, USA) (CCSW '24). Association for Computing Machinery, New York, NY, USA, 80–94. doi:10.1145/3689938.3694781
- [16] Linux kernel contributors. 2026. BPF Kernel Functions (kfuncs). <https://docs.kernel.org/bpf/kfuncs.html>. Linux kernel documentation. Accessed: 2026-06-10.
- [17] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, New York, NY, USA, 65–79. doi:10.1145/3453483.3454030
- [18] Hongyi Lu, Shuai Wang, Yechang Wu, Wanning He, and Fengwei Zhang. 2024. MOAT: Towards Safe BPF Kernel Extension. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 1153–1170. <https://www.usenix.org/conference/usenixsecurity24/presentation/lu-hongyi>
- [19] Jinsong Mao, Hailun Ding, Juan Zhai, and Shiqing Ma. 2024. Merlin: Multi-tier Optimization of eBPF Code for Performance and Compactness. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 639–653. doi:10.1145/3620666.3651387
- [20] Meta. 2018. Katran: A high performance layer 4 load balancer. <https://github.com/facebookincubator/katran>. Accessed: 2026-04-06.
- [21] Microsoft. 2026. eBPF for Windows. <https://github.com/microsoft/ebpf-for-windows>. Accessed: 2026-04-06.
- [22] George C. Necula. 1997. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Paris, France) (POPL '97). Association for Computing Machinery, New York, NY, USA, 106–119. doi:10.1145/263699.263712
- [23] Luke Nelson, Jacob Van Geffen, Emina Torlak, and Xi Wang. 2020. Specification and Verification in the Field: Applying Formal Methods to BPF Just-in-Time Compilers in the Linux Kernel. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Virtual Event, 41–61. <https://www.usenix.org>

- org/conference/osdi20/presentation/nelson
- [24] Hao Sun and Zhendong Su. 2025. Prove It to the Kernel: Precise Extension Analysis via Proof-Guided Abstraction Refinement. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) (SOSP '25). Association for Computing Machinery, New York, NY, USA, 736–751. doi:10.1145/3731569.3764796
 - [25] Dave Thaler. 2024. BPF Instruction Set Architecture (ISA). RFC 9669. doi:10.17487/RFC9669
 - [26] Jacob Van Geffen, Luke Nelson, Isil Dillig, Xi Wang, and Emina Torlak. 2020. Synthesizing JIT Compilers for In-Kernel DSLs. In *Computer Aided Verification: 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part II* (Los Angeles, CA, USA). Springer-Verlag, Berlin, Heidelberg, 564–586. doi:10.1007/978-3-030-53291-8_29
 - [27] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2023. Verifying the Verifier: eBPF Range Analysis Verification. In *Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 17–22, 2023, Proceedings, Part III* (Paris, France). Springer-Verlag, Berlin, Heidelberg, 226–251. doi:10.1007/978-3-031-37709-9_12
 - [28] Xi Wang, David Lazar, Nickolai Zeldovich, Adam Chlipala, and Zachary Tatlock. 2014. Jitk: A Trustworthy In-Kernel Interpreter Infrastructure. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 33–47. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/wang_xi
 - [29] Yiming Xiang, Wanning He, Mehbubul Hasan Al-Quvi, Emmett Witchel, and Ryan Huang. 2025. ePass: A Framework for Enhancing Flexibility and Runtime Safety of eBPF Programs. Linux Plumbers Conference (LPC). <https://lpc.events/event/19/contributions/2179/>
 - [30] Qiongwen Xu, Michael D. Wong, Tanvi Wagle, Srinivas Narayana, and Anirudh Sivaraman. 2021. Synthesizing safe and efficient kernel extensions for packet processing. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Virtual Event, USA) (SIGCOMM '21). Association for Computing Machinery, New York, NY, USA, 50–64. doi:10.1145/3452296.3472929
 - [31] Shenghao Yuan, Frédéric Besson, Jean-Pierre Talpin, Samuel Hym, Koen Zandberg, and Emmanuel Baccelli. 2022. End-to-End Mechanized Proof of an eBPF Virtual Machine for Micro-controllers. In *Computer Aided Verification: 34th International Conference, CAV 2022, Haifa, Israel, August 7–10, 2022, Proceedings, Part II* (Haifa, Israel). Springer-Verlag, Berlin, Heidelberg, 293–316. doi:10.1007/978-3-031-13188-2_15
 - [32] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 557–574. <https://www.usenix.org/conference/osdi25/presentation/zheng-yusheng>
 - [33] Qian Zhu, Yuxuan Liu, Ziyuan Zhu, Shangqing Liu, and Lei Bu. 2025. EPSO: A Caching-Based Efficient Superoptimizer for BPF Bytecode. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (Seoul, Korea, Republic of). IEEE Press, Piscataway, NJ, USA, 2945–2956. doi:10.1109/ASE63991.2025.00242